



## Question 2

**Q:** Given a real-world scene captured under low light, describe how the image sensing and acquisition process influences the final digital image.

### Theoretical Concept: Low Light Acquisition

In low-light conditions, the number of photons reaching the camera sensor (CCD/CMOS) is reduced. This leads to a low **Signal-to-Noise Ratio (SNR)**. To compensate, cameras increase ISO sensitivity, which amplifies both the light signal and the inherent electrical noise, resulting in a grainy digital image.

**Correction:** We can improve low-light images using Histogram Equalization or Gamma Correction to enhance contrast, followed by a smoothing filter to reduce noise.



### OpenCV Python Solution

Copy

```
import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for matrix operations
import matplotlib.pyplot as plt # Import Matplotlib to plot images

# Step 1: Create a synthetic image representing a clear scene
img = np.zeros((100, 100), dtype=np.uint8) # Create a blank black image array filled with zeros
cv2.circle(img, (50, 50), 30, 200, -1) # Draw a filled or outlined circle on the image

# Step 2: Simulate low light by drastically reducing contrast and brightness
low_light = cv2.convertScaleAbs(img, alpha=0.2, beta=0) # Scale and shift pixel values to simulate low light

# Step 3: Apply Histogram Equalization to mathematically stretch the contrast
enhanced = cv2.equalizeHist(low_light) # Enhance contrast using histogram equalization

# Step 4: Display the final enhanced image
plt.figure(figsize=(8,4)) # Create a new figure window for display
```

```
plt.subplot(121), plt.imshow(low_light, cmap='gray'), plt.title('Low Light Image')  
plt.subplot(122), plt.imshow(enhanced, cmap='gray'), plt.title('Enhanced via Equali  
plt.show() # Show the final plot window to the user
```

## Expected Output

The output will show a side-by-side plot. The left image will be very dark and hard to see, while the right image will have its brightness perfectly restored by the OpenCV equalization algorithm.

## How to Execute & Submit

### 1. Google Colab Execution:

- Open [Google Colab](https://colab.research.google.com/) and create a New Notebook.
- Click the  **Copy** button on the code block above.
- Paste it into a Colab cell and press **Shift + Enter** to run.

### 2. Uploading to GitHub:

- In Colab, go to **File > Save a copy in GitHub**.
- Authorize your GitHub account.
- Select your Computer Vision Lab repository and click **OK** to commit the code.